CIVL6410 Transport Network Modelling

Week 4:
Data Structures & Shortest Path Algorithm

A/Prof Jiwon Kim
Semester 1, 2026

Outline

- Network Representations and Data Structures
- Shortest Path Algorithm

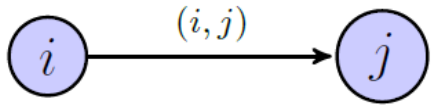
  - References
    - Bolyes, Lownes, and Unnikrishnan (BLU) Book:
      - Section 2.3 Data Structures
      - Section 2.4 Shortest Paths

  - Additional Reading
    - BLU Book: Chapter 1: Introduction to Transportation Networks

Notation and Terminology

- A ***network*** is denoted as $G = (N, A)$, where N is the set of nodes and A is the set of links.

- A ***link*** connecting node i and node j is denoted as (i, j):



  - Node i is called the '*tail*', 'upstream node', or 'initial node'
  - Node j is called the '*head*', *'downstream node', or 'terminal node'*

- Example:
  - $N = \{1, 2, 3, 4\}$
  - $A = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4)\}$

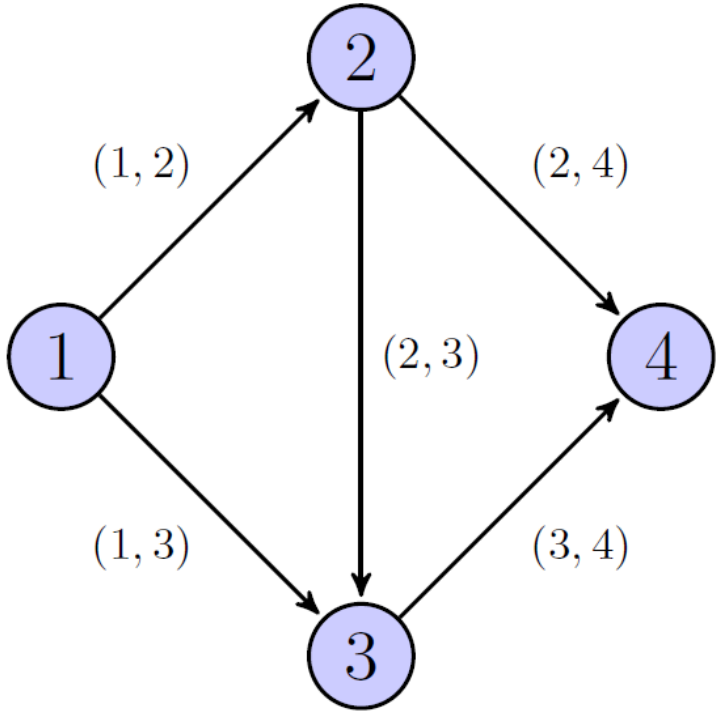
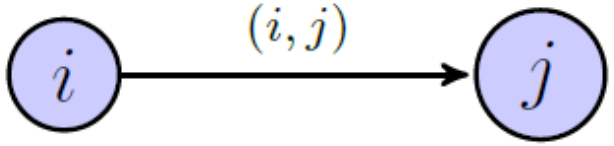


Figure 2.1: Example network notation

Notation and Terminology



- Link (i, j) is an ***outgoing*** link of node i.
- Link (i, j) is an ***incoming*** link of node j.

- Example
  - (2,3) and (2,4) are outgoing links of node 2.
  - (1,2) is an incoming link of node 2.

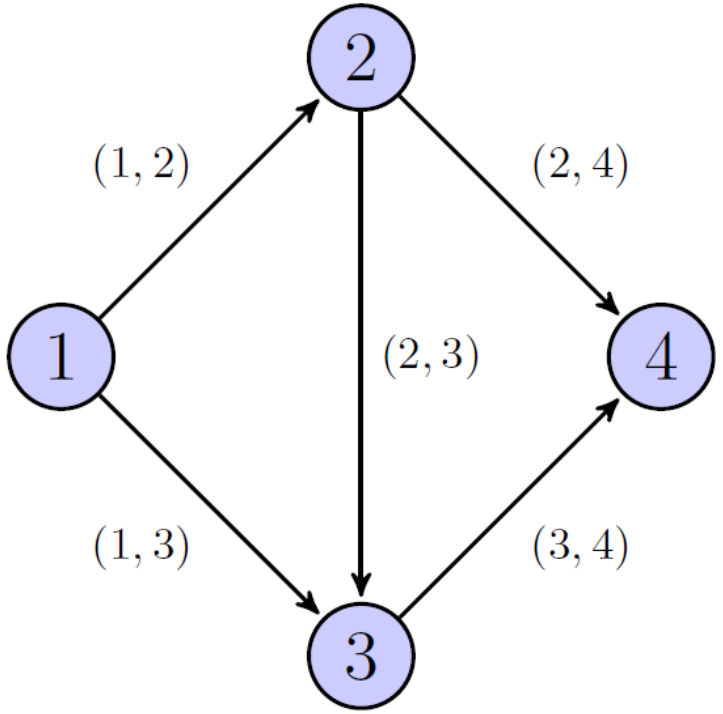


Figure 2.1: Example network notation

Forward Star and Reverse Star

- The ***forward star*** of a node i is the set of all outgoing (downstream) links, denoted by $\Gamma(i)$.
- The ***reverse star*** of a node i is the set of all incoming (upstream) links, denoted by $\Gamma^{-1}(i)$.

- Example
  - The *forward star* and *reverse star* of node 2 are $\Gamma(2) = \{(2,3),(2,4)\}$ and $\Gamma^{-1}(2) = \{(1,2)\}$, respectively.
  - The *forward star* and *reverse star* of node 3 are $\Gamma(3) = \{(3,4)\}$ and $\Gamma^{-1}(3) = \{(1,3),(2,3)\}$, respectively.

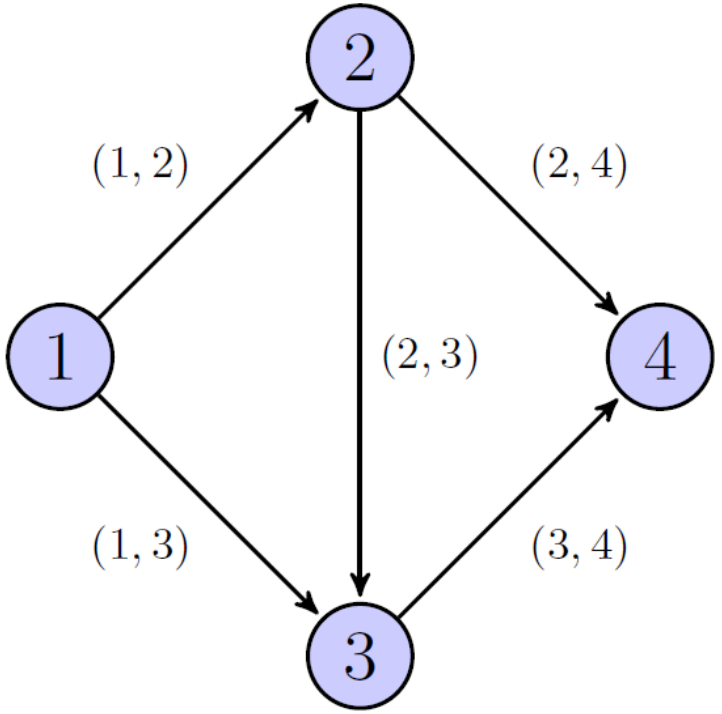


Figure 2.1: Example network notation

Degree, Indegree, and Outdegree

- The ***degree*** of a node is the total number of links connected to that node.
- The ***indegree*** is the total number of incoming links at a node (i.e., the size of the reverse star, $|\Gamma^{-1}(i)|$).
- The ***outdegree*** is the total number of outgoing links at a node (i.e., the size of the forward star, $|\Gamma(i)|$).
- The degree is the sum of the indegree and outdegree.

- Example
  - Node 2 has an indegree of 1, an outdegree of 2, and a degree of 3.

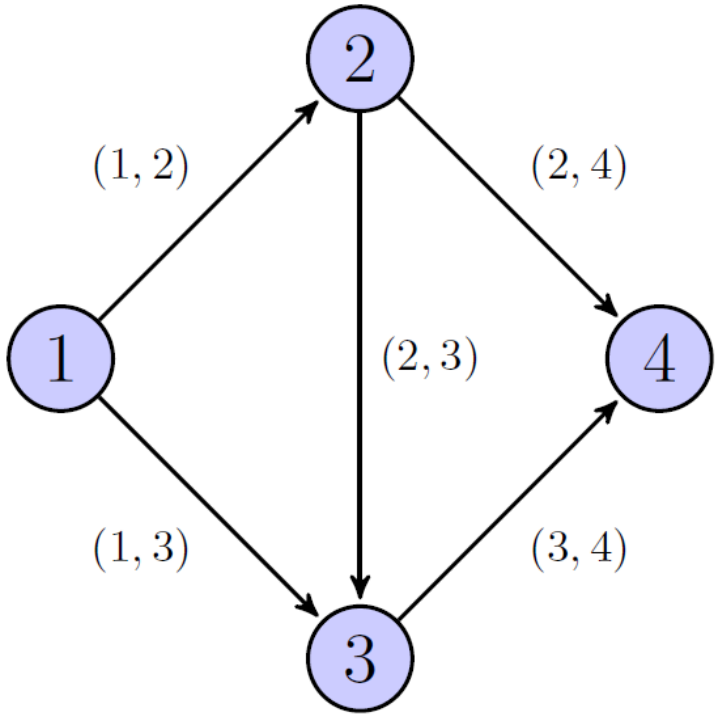


Figure 2.1: Example network notation

Node-Link Incident Matrix

- A matrix with **rows** representing **nodes** and **columns** representing **links**, where entry at (i, j) is assigned:
  - 1: when the j^{th} link is an outgoing link from the i^{th} node
  - -1: when the j^{th} link is an incoming link to the i^{th} node.
  - 0: when the j^{th} link is not incident to the i^{th} node.

$$
\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array}
\begin{array}{c}
\begin{array}{ccccc} (1,2) & (1,3) & (2,3) & (2,4) & (3,4) \end{array} \\
\left[\begin{array}{ccccc}
1 & 1 & 0 & 0 & 0 \\
-1 & 0 & 1 & 1 & 0 \\
0 & -1 & -1 & 0 & 1 \\
0 & 0 & 0 & -1 & -1
\end{array}\right]
\end{array}
$$

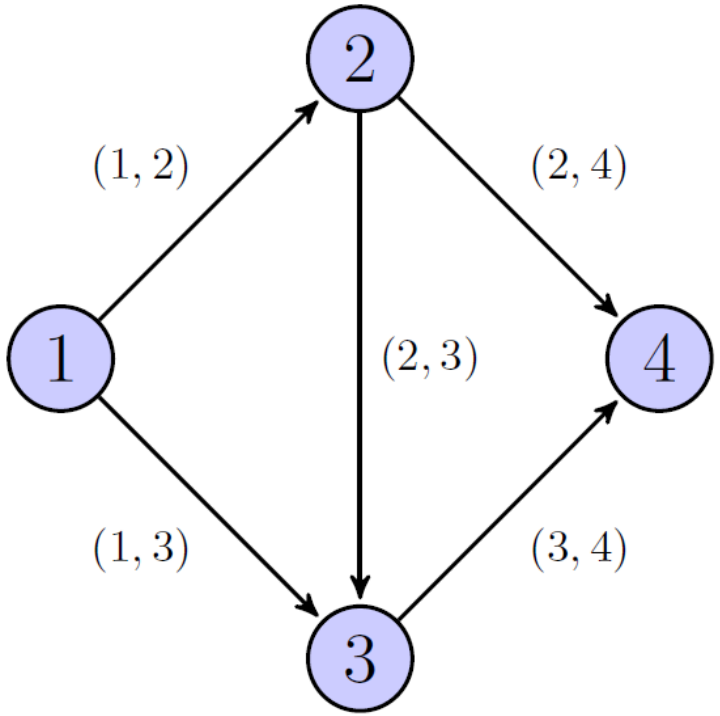


Figure 2.1: Example network notation

Node-Node Adjacency Matrix

- A matrix with both **rows** and **columns** representing **nodes**, where entry at (i, j) is assigned:
  - 1: when there is a link from the i^{th} node to the j^{th} node
  - 0: when there is no link from the i^{th} node to the j^{th} node

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

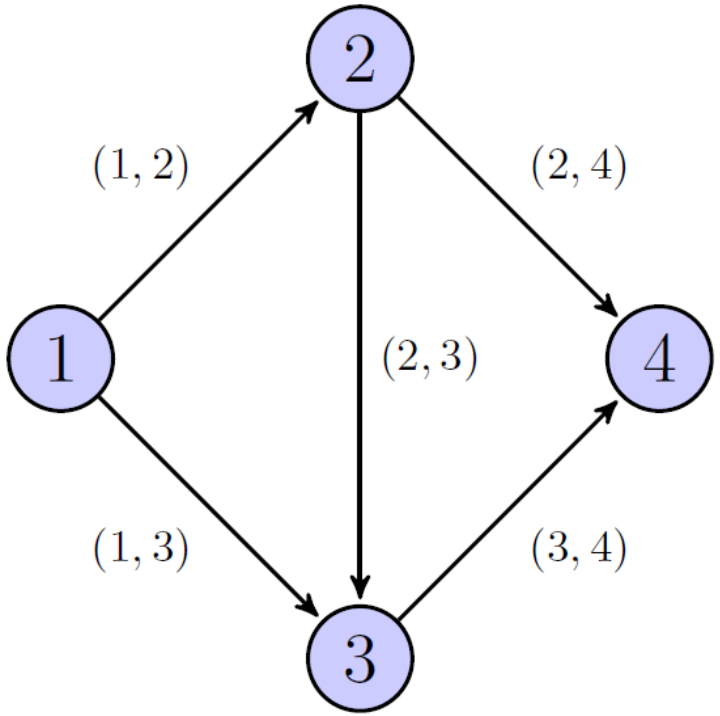
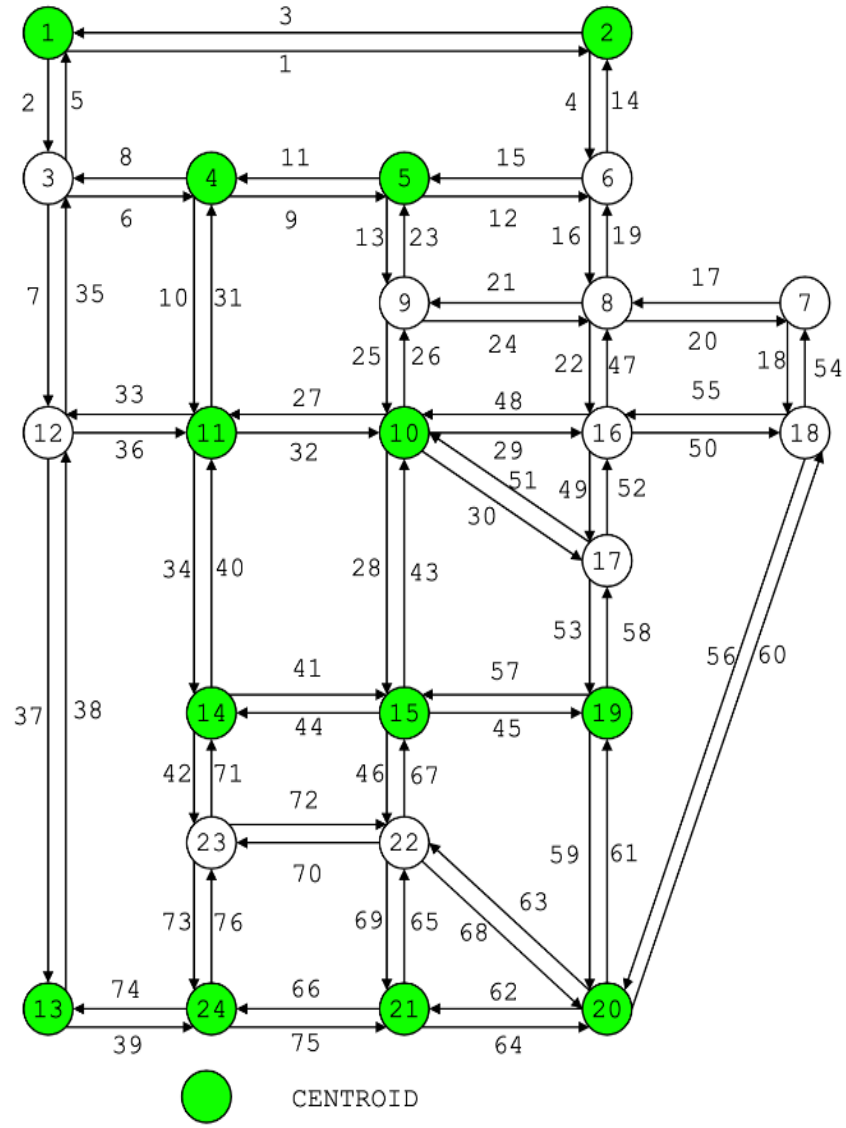


Figure 2.1: Example network notation

Storage Comparison

- Node-Link Incident Matrix: #nodes × #links
  - 24 × 76 = 1824
- Node-Node Adjacency Matrix: #nodes × #nodes
  - 24 × 24 = 576



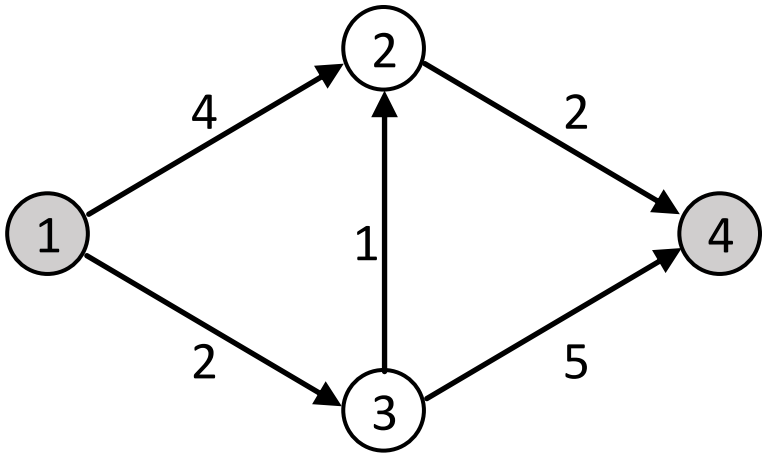
Sioux-Falls network
#links = 76 (number of links)
#nodes = 24 (number of nodes)

Shortest Path Problem

Shortest Path Problem

- Given a network $G = (N, A)$, where each link $(i, j) \in A$ has a *fixed* cost c_{ij}, find the shortest path incurring the least total **cost** (e.g., travel time, toll, fuel cost, etc.).

- What is the shortest path from node 1 to node 4?
  - The cost of Path [1,2,4] = 4+2 = 6
  - The cost of Path [1,3,4] = 2+5 = 7
  - The cost of Path [1,3,2,4]= 2+1+2 = 5
  - **The shortest path from 1 to 4 is Path [1,3,2,4].**



Adapted from [BLU: Fig 2.10]
Example network for shortest paths
(link costs indicated)

Shortest Path Problem

- Given a network $G = (N, A)$, where each link $(i, j) \in A$ has a ***fixed*** cost c_{ij}, find the shortest path incurring the least total cost (e.g., travel time, toll, fuel cost, etc.).

- Why assuming **"fixed"** costs?
  - In many transport problems, the costs are not fixed, but rather dynamic:
    - The cost depends on the total flow and vehicle route choices .
  - However, even in such problems, the most practical solution methods involve **solving several shortest path problems** as **part of an *iterative* framework**, updating paths as travel times change.
  - As such, a shortest path algorithm focuses on **finding the least travel-time path** between two nodes, **at the current travel times**—with everybody's route choices held fixed.

Bellman's Optimality Principle

- *A Key Concept in 'Dynamic Programming'*

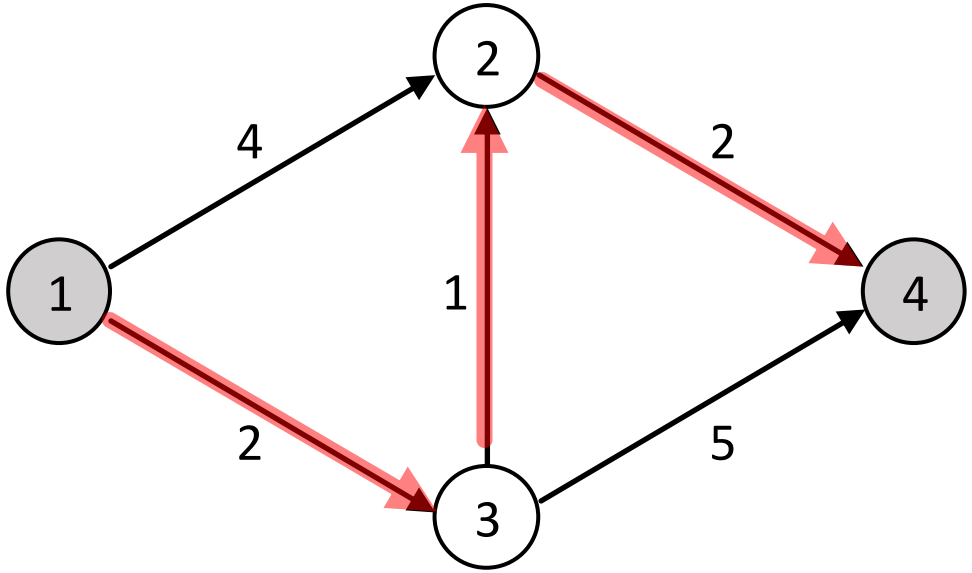
- Bellman's principle, in general, states that:
  - **"Any optimal solution to a problem is made up of smaller sub-solutions that are also optimal."**
    - The best overall strategy is made up of the best smaller steps.
    - If you're making the best choices at every step, you're on the best path overall.

- How it relates to shortest path:
  - **"Any segment of a shortest path must itself be a shortest path between its endpoints."**
    - A fundamental concept used in many shortest path algorithms

Bellman's Optimality Principle

- A fundamental concept used in many shortest path algorithms: "***Any segment of a shortest path must itself be a shortest path between its endpoints***."

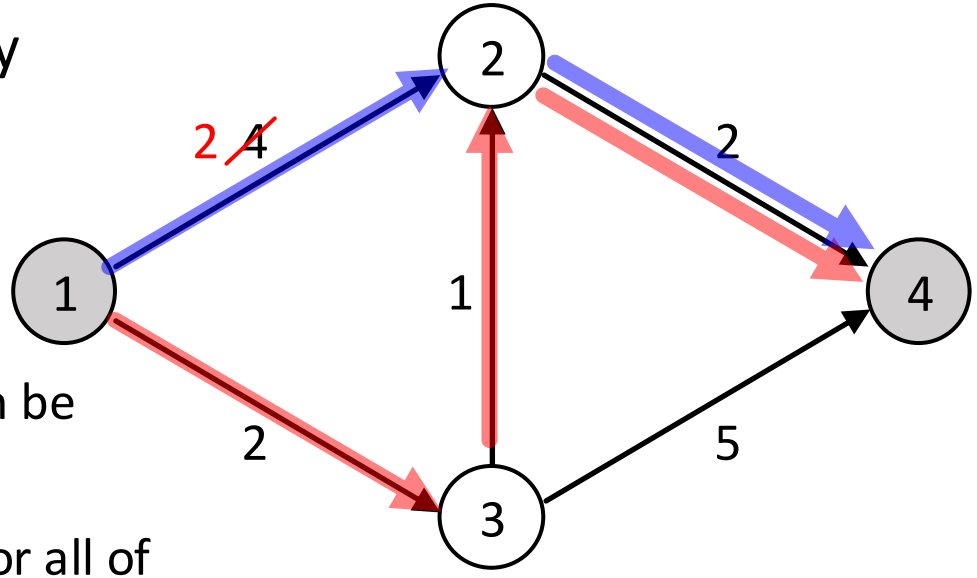
- If Path [1,3,2,4] is the shortest path from 1 to 4,
  - Path [1,3,2] is the shortest path from 1 to 2
  - Path [3,2,4] is the shortest path from 3 to 4
  - Path [1,3] is the shortest path from 1 to 3
  - Path [3,2] is the shortest path from 3 to 2
  - Path [2,4] is the shortest path from 2 to 4



Bellman's Optimality Principle

- A fundamental concept used in many shortest path algorithms: "***Any segment of a shortest path must itself be a shortest path between its endpoints***."

- Can there be the shortest path that does not satisfy Bellman's Principle for any of its segments?
  - Suppose the cost on link (1,2) is reduced to 2.
  - The shortest path from 1 to 2 is now [1,2], not [1,3,2].
  - Then, the cost of the entire shortest path from 1 to 4, can be reduced by changing from [1,3,2,4] to [1,2,4].
  - Thus, the shortest path must satisfy Bellman's Principle for all of its segments.



Bellman's Optimality Principle

- The implication: "**We can construct shortest paths on one node at a time, proceeding inductively.**"
- To find the shortest path from node r to node i, instead of checking all possible paths between r and i, we only need to check:
  - The shortest known paths from r to node i's immediate upstream nodes, i.e., f, g, and h (already computed)
  - The direct edges from f, g, and h to node i

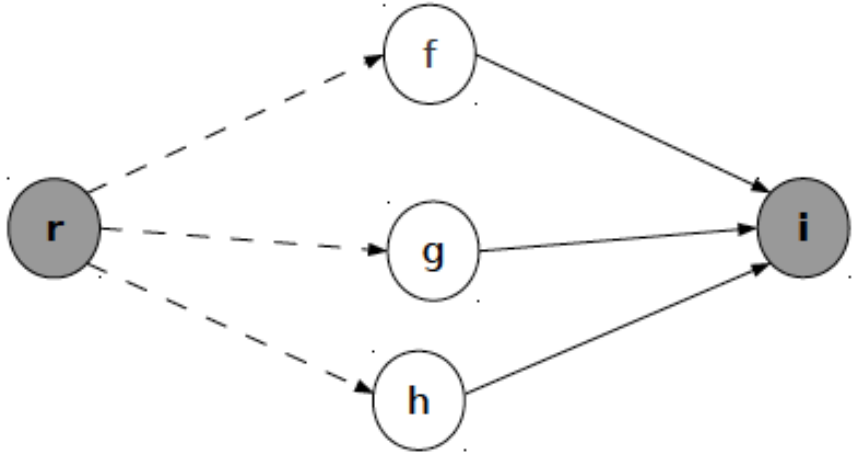


Figure 2.11: Illustration of Bellman's Principle (dashed lines indicated shortest paths between two nodes).

Data Structures

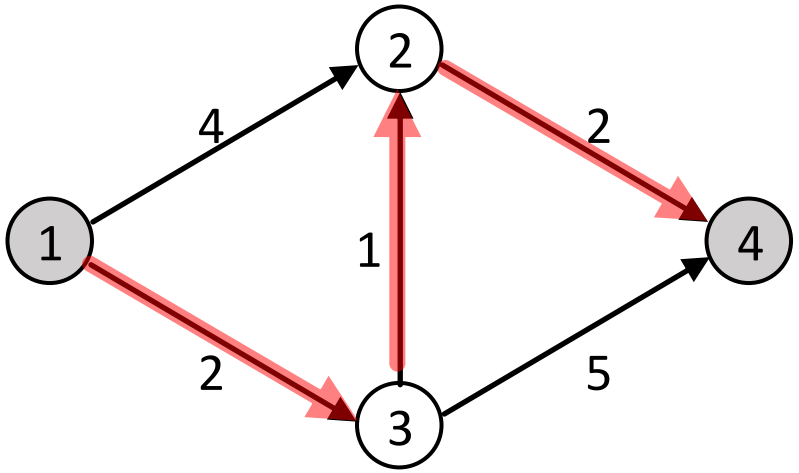
- Given a network $G = (N, A)$, the shortest paths from origin node r to every other node can be stored in the following two vectors: ***Backnode*** vector $\mathbf{q}^r$ and ***Label*** vector $\mathbf{L}^r$.

- **Backnode Vector: $\mathbf{q}^r = [q_1^r, q_2^r, \dots, q_i^r, \dots, q_{|N|}^r]$**
  - A vector of length $|N|$, where $|N|$ is the number of nodes in graph G
  - q_i^r indicates the *backnode* of node i (the node immediately preceding node i) in the shortest path from origin r to node i
  - If $i = r$, we define $q_r^r \equiv -1$, e.g., the *backnode* is the origin node is set to -1.

Data Structures

- **Backnode Vector**: $\mathbf{q}^r = [q_1^r, q_2^r, \dots, q_i^r, \dots, q_{|N|}^r]$
  - A vector of length $|N|$, where $|N|$ is the number of nodes in graph G
  - q_i^r indicates the *backnode* of node i (the node immediately preceding node i) in the shortest path from origin r to node i
  - If $i = r$, we define $q_r^r \equiv -1$, e.g., the *backnode* is the origin node is set to -1.

- Example
  - The shortest path from 1 to 4 is [1, 3, 2, 4].
  - Then the backnode vector is constructed as:
  - $\mathbf{q}^1 = [q_1^1, q_2^1, q_3^1, q_4^1] = [-1, 3, 1, 2]$



Data Structures

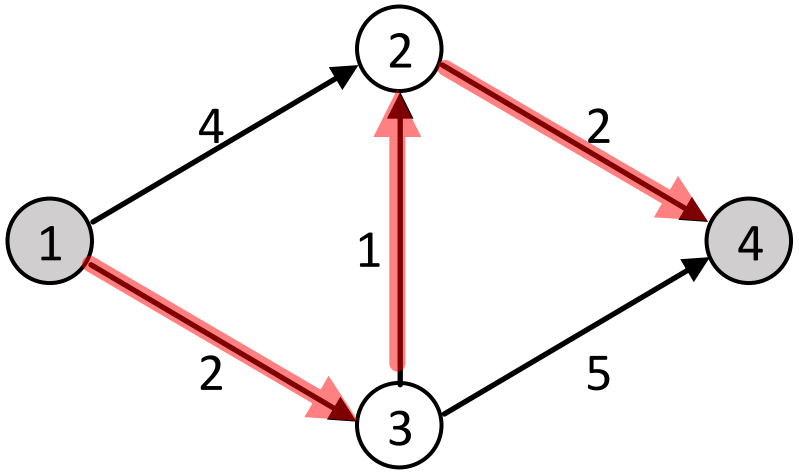
- **Backnode Vector**: $\mathbf{q}^r = [q_1^r, q_2^r, \dots, q_i^r, \dots, q_{|N|}^r]$
  - A vector of length $|N|$, where $|N|$ is the number of nodes in graph G
  - q_i^r indicates the *backnode* of node i (the node immediately preceding node i) in the shortest path from origin r to node i
  - If $i = r$, we define $q_r^r \equiv -1$, e.g., the *backnode* is the origin node is set to -1.

- **Label Vector**: $\mathbf{L}^r = [L_1^r, L_2^r, \dots, L_i^r, \dots, L_{|N|}^r]$
  - A vector of length $|N|$, where $|N|$ is the number of nodes in graph G
  - L_i^r indicates the *total cost* on the shortest path from origin r to node i
  - If $i = r$, we define $L_r^r \equiv 0$, e.g., the *shortest path cost* at the origin node is set to 0.

Data Structures

- **Label Vector**: $\mathbf{L}^r = [L_1^r, L_2^r, \dots, L_i^r, \dots, L_{|N|}^r]$
  - A vector of length $|N|$, where $|N|$ is the number of nodes in graph G
  - L_i^r indicates the *total cost* on the shortest path from origin r to node i
  - If $i = r$, we define $L_r^r \equiv 0$, e.g., the *shortest path cost* at the origin node is set to 0.

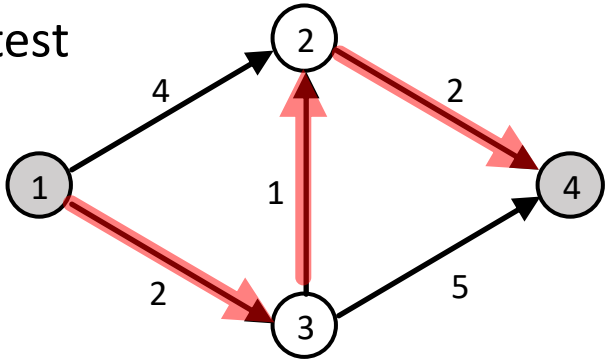
- Example
  - The shortest path from 1 to 4 is [1, 3, 2, 4].
  - Then the backnode vector and label vector are constructed as:
  - $\mathbf{q}^1 = [q_1^1, q_2^1, q_3^1, q_4^1] = [-1, 3, 1, 2]$
  - $\mathbf{L}^1 = [L_1^1, L_2^1, L_3^1, L_4^1] = [0, 3, 2, 5]$



Use of Backnode Vector

- When the shortest path algorithm returns $\mathbf{q}^r$ as output, it can be used to track the shortest path back to an origin by starting from the destination, and reading back one node at a time.

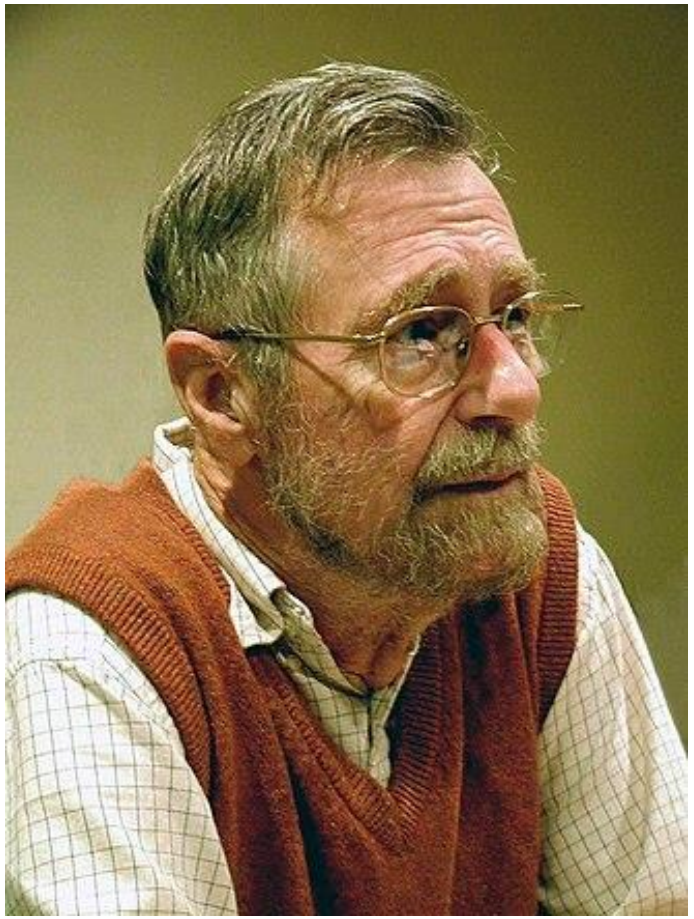
- To look up the shortest path from node 1 to node 4, start from the destination 4 and read $\mathbf{q}^1 = [q_1^1, q_2^1, q_3^1, q_4^1] = [-1, 3, 1, 2]$ as follows:
  - $q_4^1 = 2$ (the backnode of 4 is 2) "the shortest path to node 4 is the shortest path to node 2, plus link (2, 4).
  - $q_2^1 = 3$ (the backnode of 2 is 3): the shortest path to node 2 is the shortest path to node 3, plus link (3, 2).
  - $q_3^1 = 1$ (the backnode of 3 is 1): the shortest path to node 3 is the shortest path to node 1, plus link (1, 3).
  - $q_1^1 = -1$: node 1 is the origin.
- The shortest path is: 4 ← 2 ← 3 ← 1 , i.e., [1, 3, 2, 4]



Dijkstra’s Shortest Path Algorithm

Dijkstra's Shortest Path Algorithm

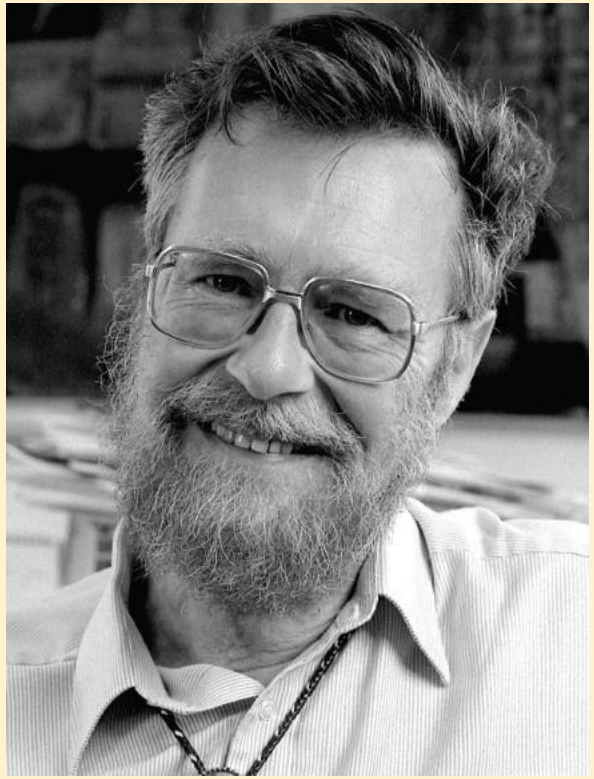
- Developed by a Dutch computer scientist, ***Edsger Wybe Dijkstra*** (/ˈdaɪkstrə/), in 1956
- Finds the shortest path from **a single origin** node to **every other node** in the network.
  - It uses ***Bellman's Principle***, i.e., the shortest path from *r* to *s* must also contain the shortest path from *r* to every node in that path.
- It assumes all links have **non-negative link costs** (weights).
- It can be applied to **a network with cycles** (as long as the network does not have negative cycles)



Edsger Wybe Dijkstra (1930–2002)
(source: Wikipedia)

Dijkstra's Shortest Path Algorithm

What's the shortest way to travel from Rotterdam to Groningen? It is the algorithm for the shortest path, which I designed in about 20 minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a 20-minute invention. In fact, it was published in 1959, three years later. The publication is still quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. Without pencil and paper you are almost forced to avoid all avoidable complexities. Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame. I found it in the



From "**An Interview with Edsger W. Dijkstra**"
(one of his last interviews conducted in 2001, before his death in 2002)
https://dl.acm.org/doi/10.1145/1787234.1787249

Dijkstra's Shortest Path Algorithm

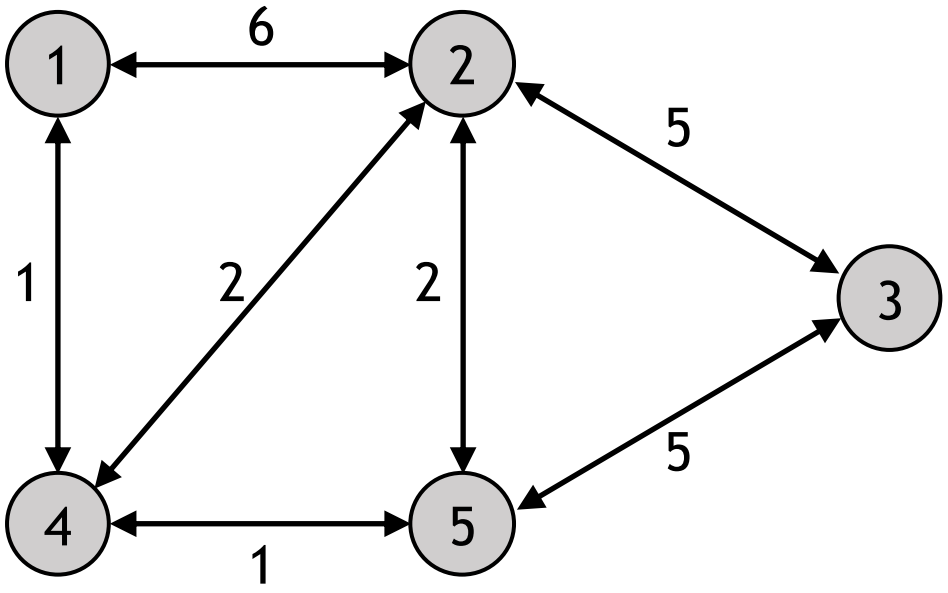
- Find the shortest path from origin r to every other node.
- For each node, assign some initial shortest path ($\mathbf{q}^r$) and shortest path cost ($\mathbf{L}^r$)
- For each node i (starting from $i = r$), check if the shortest path from r to its downstream node j should pass link (i, j):
  - Check if the sum of the shortest path cost from r to i and the link cost on (i, j) is smaller than the current shortest path cost from r to j, i.e., $L_i^r + t_{ij} < L_j^r$.
- If so, node i becomes the new backnode of j and the current shortest path cost from r to j is updated to $L_i^r + t_{ij}$; so $\mathbf{q}^r$ and $\mathbf{L}^r$ are updated accordingly.
- The process continues until all nodes are evaluated.

Node i	L_i^r (current shortest path cost from *r to i*)	q_i^r (backnode of *i* in the current shortest path)
1		
2		
3		
⋮		
N		

Q = { }
Q: the set of unvisited nodes

Dijkstra's Shortest Path Algorithm

- Find the shortest path from node 1 to every other node.

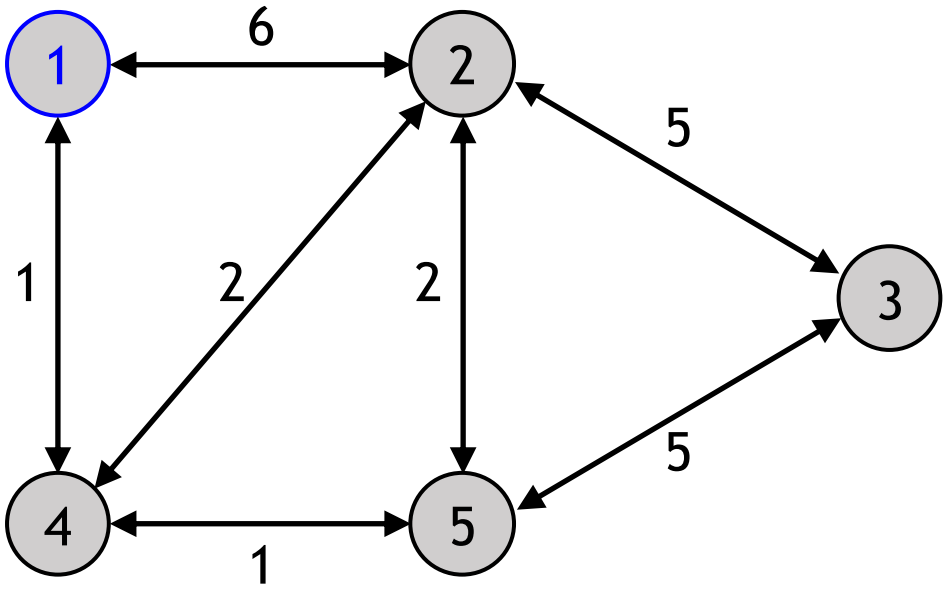


Node i	L_i^1 (current shortest path cost from *r to i*)	q_i^1 (backnode of *i* in the current shortest path)
1		
2		
3		
4		
5		

$Q = \{\ \}$
Q: the set of unvisited nodes

Dijkstra's Shortest Path Algorithm – Step 1

- Initialise every label L_i^r to ∞ , except for the origin. For the origin, set $L_r^r \leftarrow 0$.
- Initialise the backnode vector $\mathbf{q}^r \leftarrow -1$.
- Add all nodes to the set of *unvisited* nodes, Q.

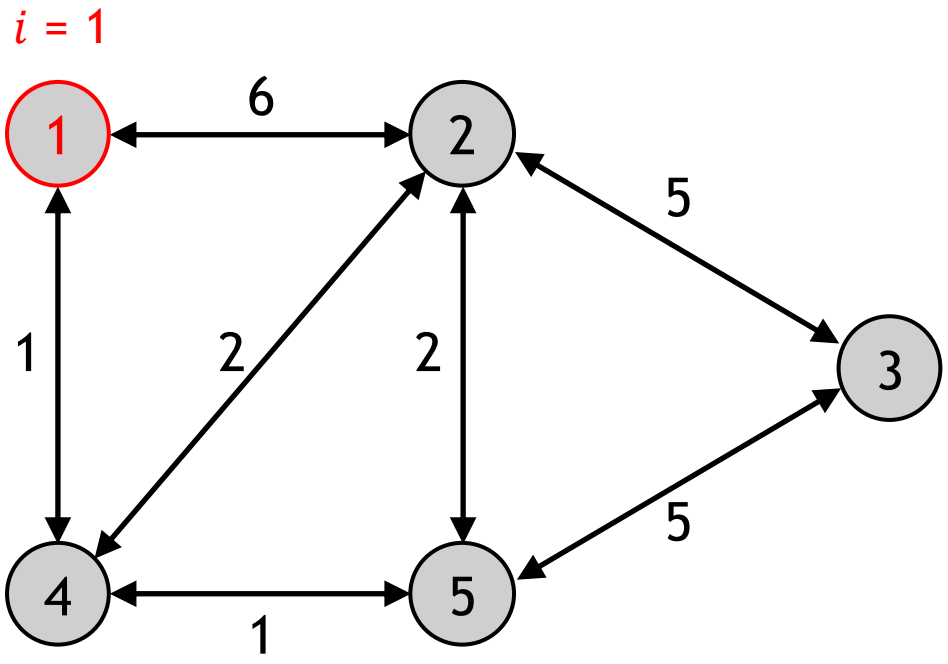


Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	∞	-1
3	∞	-1
4	∞	-1
5	∞	-1

Q = { 1, 2, 3, 4, 5 }
Q: the set of unvisited nodes

Dijkstra's Shortest Path Algorithm – Step 2

- Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{i \in Q}{\operatorname{argmin}} L_i^r$
- Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$



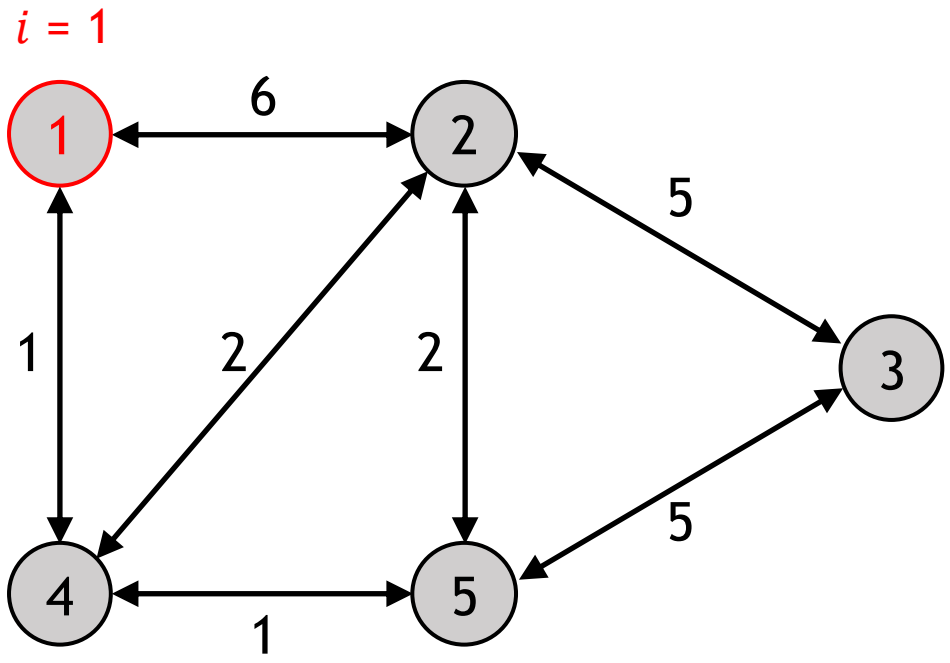
Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	∞	-1
3	∞	-1
4	∞	-1
5	∞	-1

Q = { 2, 3, 4, 5 }

Dijkstra's Shortest Path Algorithm – Step 3

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



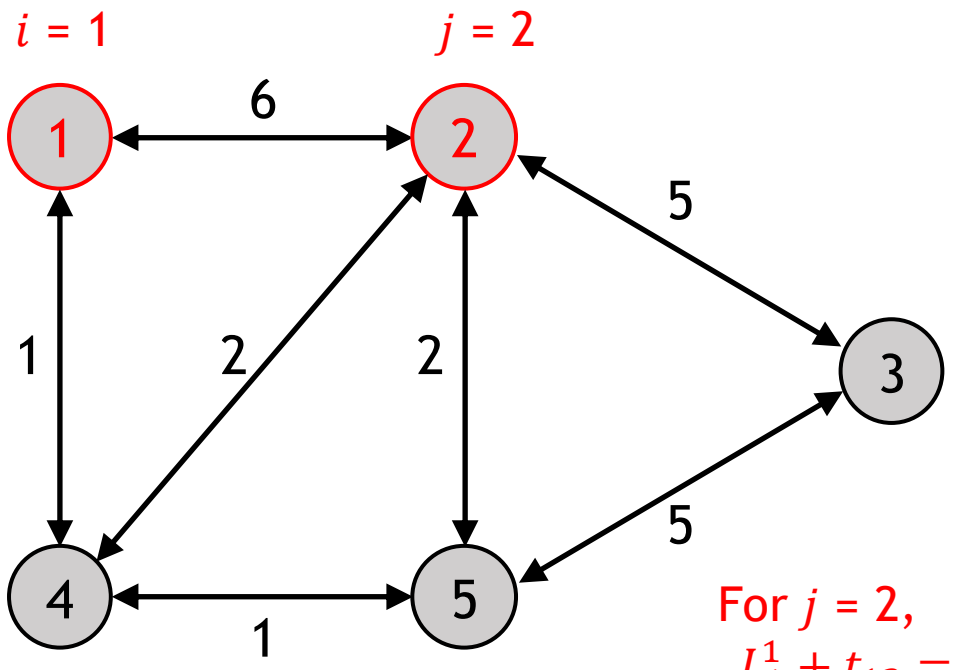
Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	∞	-1
3	∞	-1
4	∞	-1
5	∞	-1

$Q = \{ 2, 3, 4, 5 \}$

Dijkstra's Shortest Path Algorithm – Step 3

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	6	1
3	∞	-1
4	∞	-1
5	∞	-1

For $j = 2$,

$L_1^1 + t_{12} = 0 + 6 = 6 < L_2^1 = \infty$

$L_2^1 \leftarrow 6$

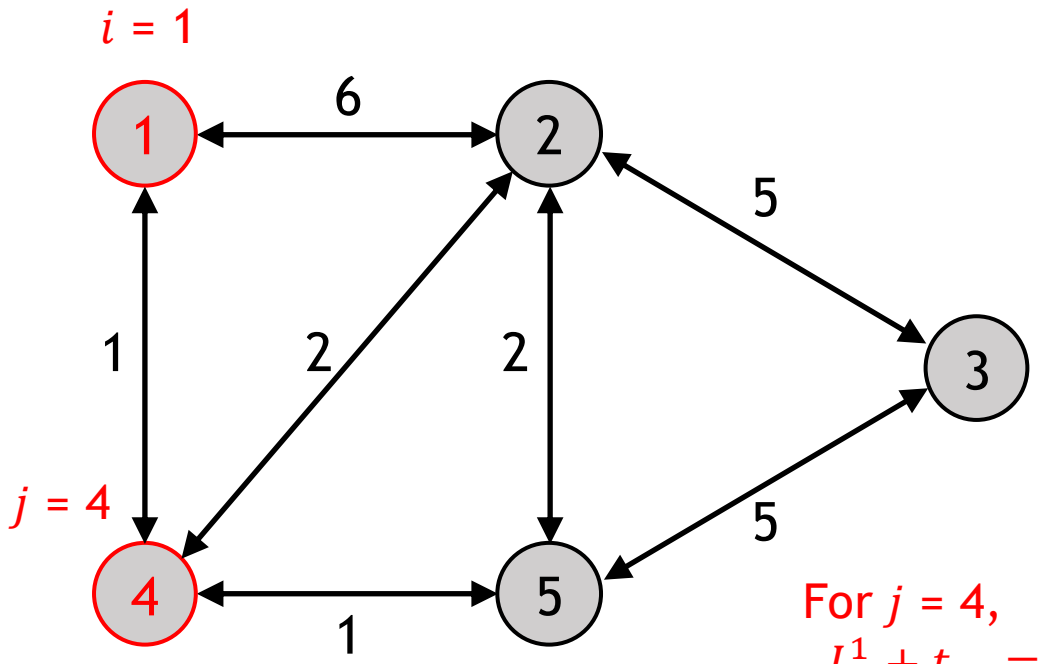
$q_2^1 \leftarrow 1$

$Q = \{ 2, 3, 4, 5 \}$

Dijkstra's Shortest Path Algorithm – Step 3

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	6	1
3	∞	-1
4	1	1
5	∞	-1

For $j = 4$,

$L_1^1 + t_{14} = 0 + 1 = 1 < L_4^1 = \infty$

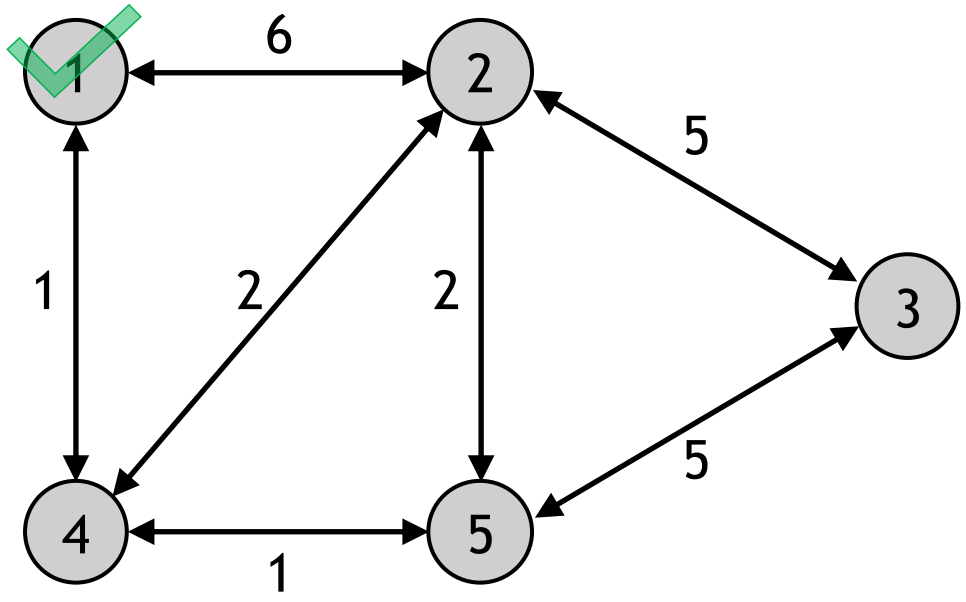
$L_4^1 \leftarrow 1$

$q_4^1 \leftarrow 1$

$Q = \{ 2, 3, 4, 5 \}$

Dijkstra's Shortest Path Algorithm – Step 4

- If the unvisited node set, Q, is empty (all nodes are finalised), stop. Return $\mathbf{L}^r$ and $\mathbf{q}^r$.
- Otherwise, return to Step 2.

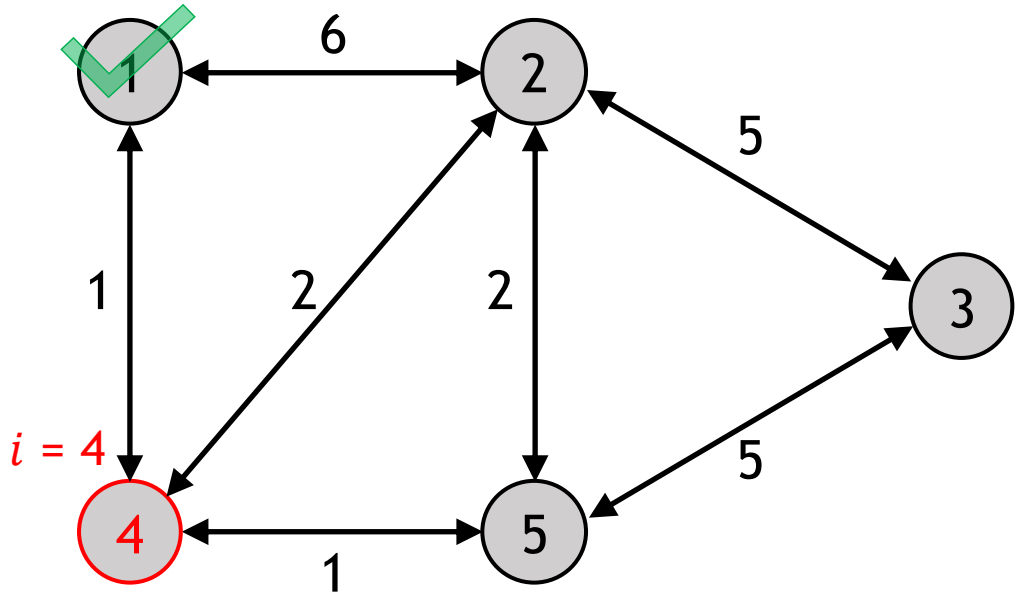


Node i	L_i^1 (current shortest path cost from *r to i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	6	1
3	∞	-1
4	1	1
5	∞	-1

$Q = \{ 2, 3, 4, 5 \} \neq \emptyset$
Return to Step 2

Dijkstra's Shortest Path Algorithm – Step 2 (2)

- Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{i \in Q}{\operatorname{argmin}} L_i^r$
- Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$



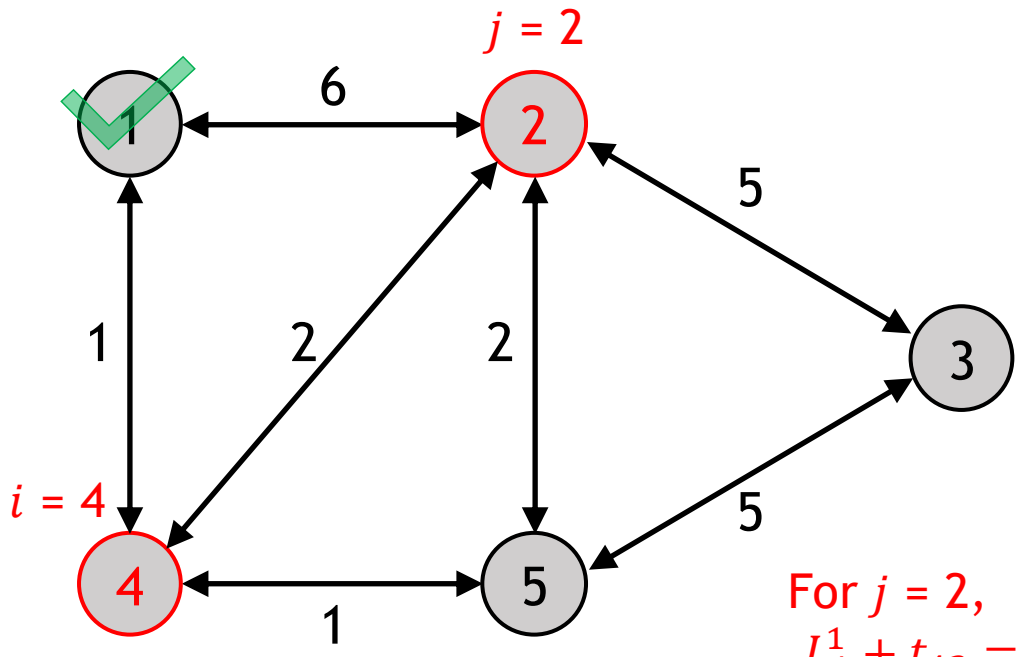
Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	6	1
3	∞	-1
4	1	1
5	∞	-1

$Q = \{ 2, 3, 5 \}$

Dijkstra's Shortest Path Algorithm – Step 3 (2)

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	∞	-1
4	1	1
5	∞	-1

For j = 2,

$L_4^1 + t_{42} = 1 + 2 = 3 < L_2^1 = 6$

$L_2^1 \leftarrow 3$

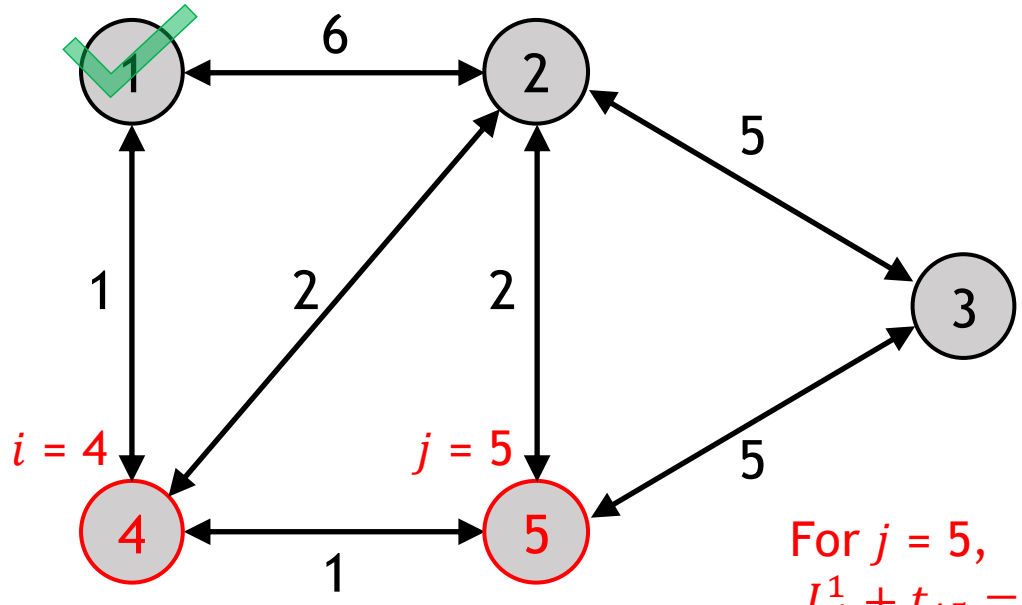
$q_2^1 \leftarrow 4$

Q = { 2, 3, 5 }

Dijkstra's Shortest Path Algorithm – Step 3 (2)

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	2
3	∞	-1
4	1	1
5	2	4

For $j = 5$,

$L_4^1 + t_{45} = 1 + 1 = 2 < L_5^1 = \infty$

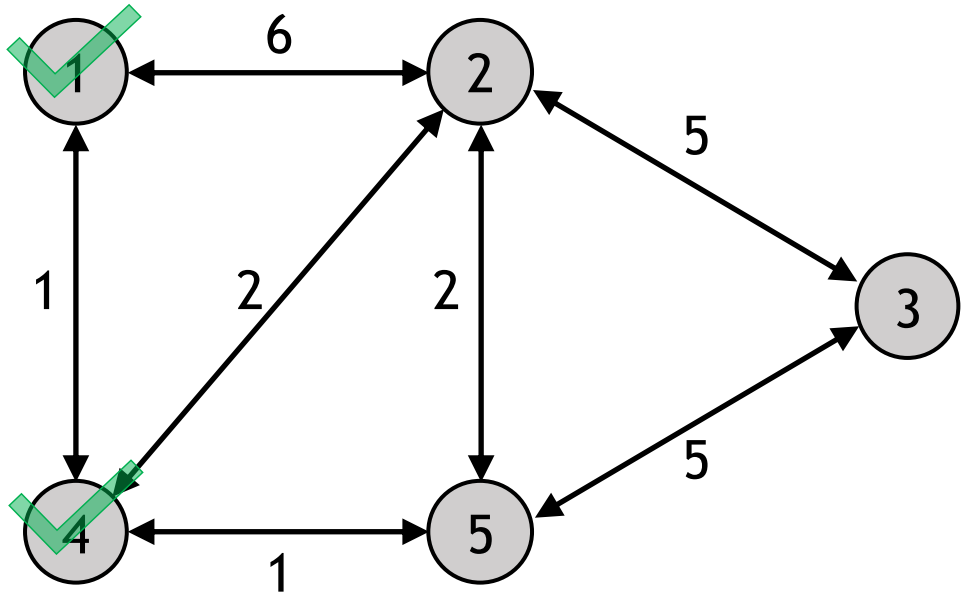
$L_5^1 \leftarrow 2$

$q_5^1 \leftarrow 4$

$Q = \{ 2, 3, 5 \}$

Dijkstra's Shortest Path Algorithm – Step 4 (2)

- If the unvisited node set, Q, is empty (all nodes are finalised), stop. Return $\mathbf{L}^r$ and $\mathbf{q}^r$.
- Otherwise, return to Step 2.

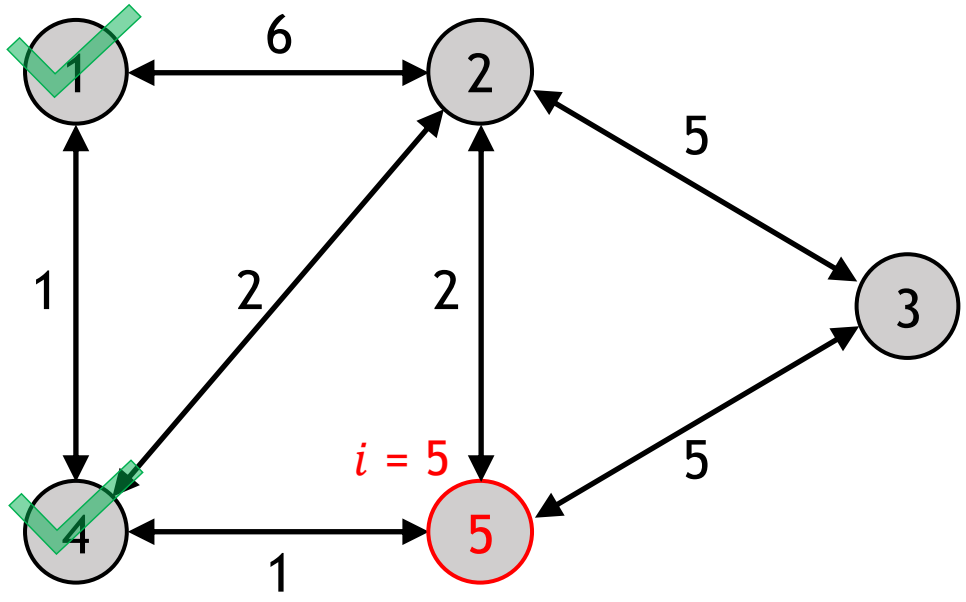


Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	∞	-1
4	1	1
5	2	4

$Q = \{ 2, 3, 5 \} \neq \emptyset$
Return to Step 2

Dijkstra's Shortest Path Algorithm – Step 2 (3)

- Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{i \in Q}{\operatorname{argmin}} L_i^r$
- Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$



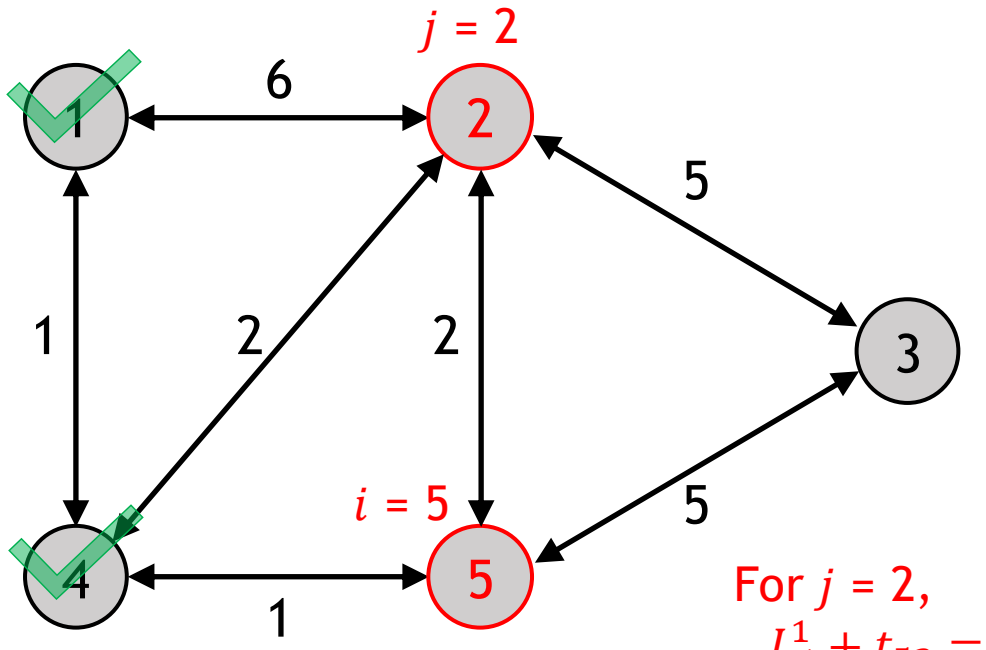
Node i	L_i^1 (current shortest path cost from *r to i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	∞	-1
4	1	1
5	2	4

Q = { 2, 3 }

Dijkstra's Shortest Path Algorithm – Step 3 (3)

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	∞	-1
4	1	1
5	2	4

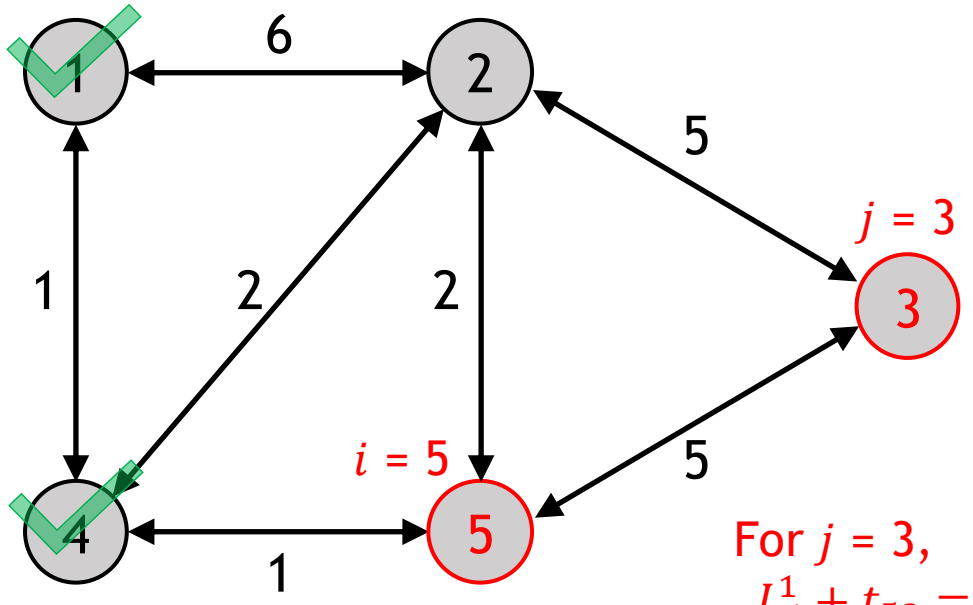
For j = 2,
$L_5^1 + t_{52} = 2 + 2 = 4 > L_2^1 = 3$
So, we don't update L_2^1 and q_2^1.

Q = { 2, 3 }

Dijkstra's Shortest Path Algorithm – Step 3 (3)

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

For j = 3,

$L_5^1 + t_{53} = 2 + 5 = 7 < L_3^1 = \infty$

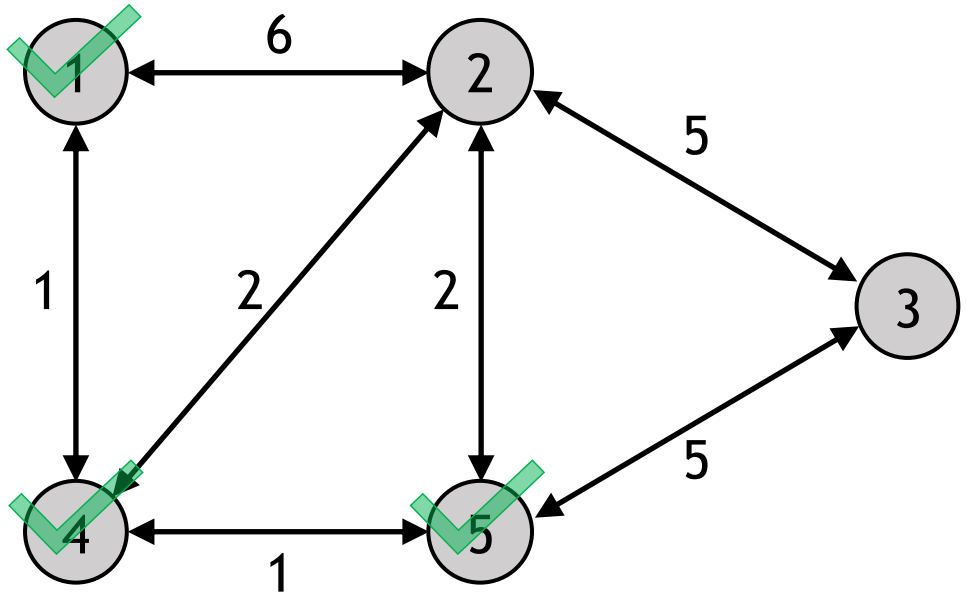
$L_3^1 \leftarrow 7$

$q_3^1 \leftarrow 5$

Q = { 2, 3 }

Dijkstra's Shortest Path Algorithm – Step 4 (3)

- If the unvisited node set, Q, is empty (all nodes are finalised), stop. Return $\mathbf{L}^r$ and $\mathbf{q}^r$.
- Otherwise, return to Step 2.

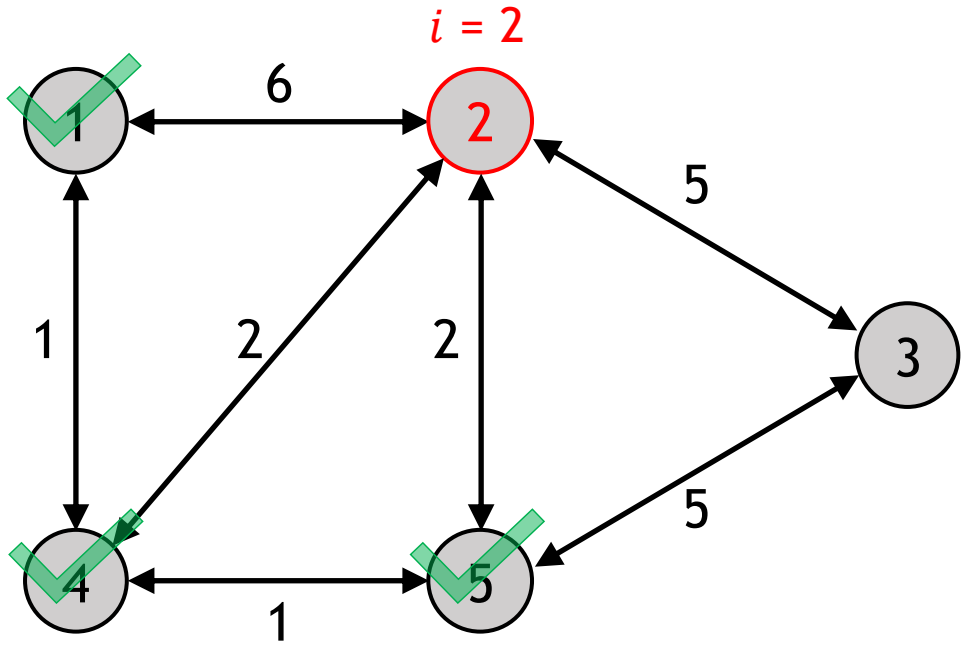


Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

$Q = \{ 2, 3 \} \neq \emptyset$
Return to Step 2

Dijkstra's Shortest Path Algorithm – Step 2 (4)

- Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{i \in Q}{\operatorname{argmin}} L_i^r$
- Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$



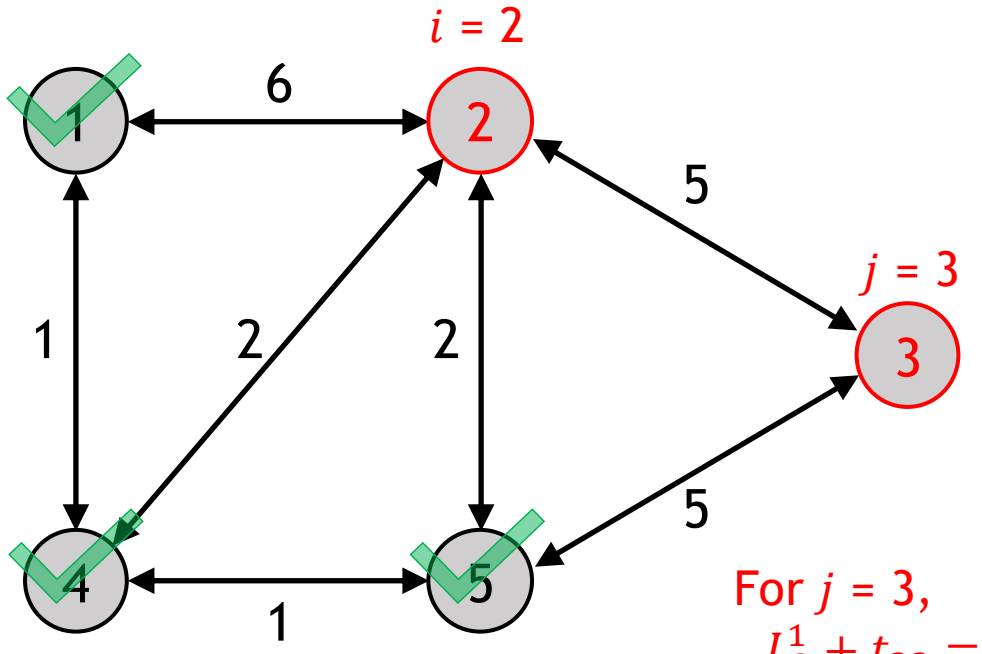
Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

$Q = \{ 3 \}$

Dijkstra's Shortest Path Algorithm – Step 3 (4)

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



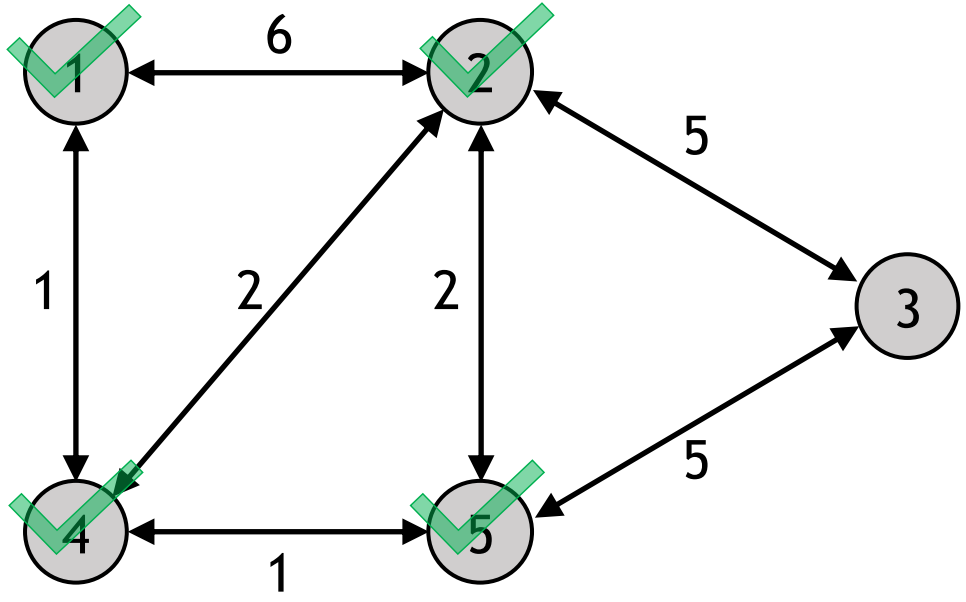
For j = 3,
$L_2^1 + t_{23} = 3 + 5 = 8 > L_3^1 = 7$
So, we don't update L_3^1 and q_3^1.

Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

Q = { 3 }

Dijkstra's Shortest Path Algorithm – Step 4 (4)

- If the unvisited node set, Q, is empty (all nodes are finalised), stop. Return $\mathbf{L}^r$ and $\mathbf{q}^r$.
- Otherwise, return to Step 2.

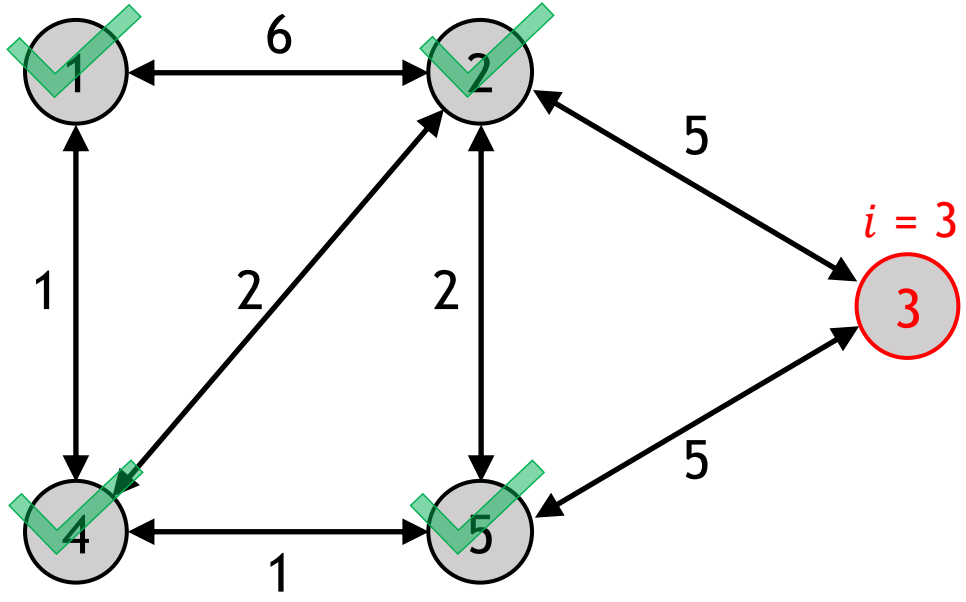


Node i	L_i^1 (current shortest path cost from *r to i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

$Q = \{ 3 \} \neq \emptyset$
Return to Step 2

Dijkstra's Shortest Path Algorithm – Step 2 (5)

- Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{i \in Q}{\operatorname{argmin}} L_i^r$
- Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$



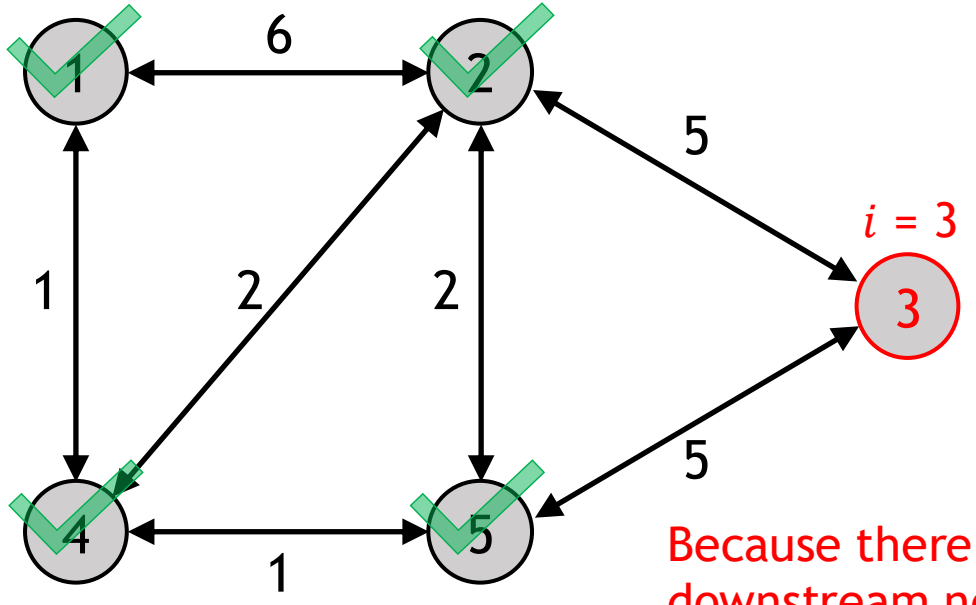
Node i	L_i^1 (current shortest path cost from *r to i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

$Q = \{\ \}$

Dijkstra's Shortest Path Algorithm – Step 3 (5)

- For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows:

$$\text{If } L_i^r + t_{ij} < L_j^r,$$
$$L_j^r \leftarrow L_i^r + t_{ij}$$
$$q_j^r \leftarrow i$$



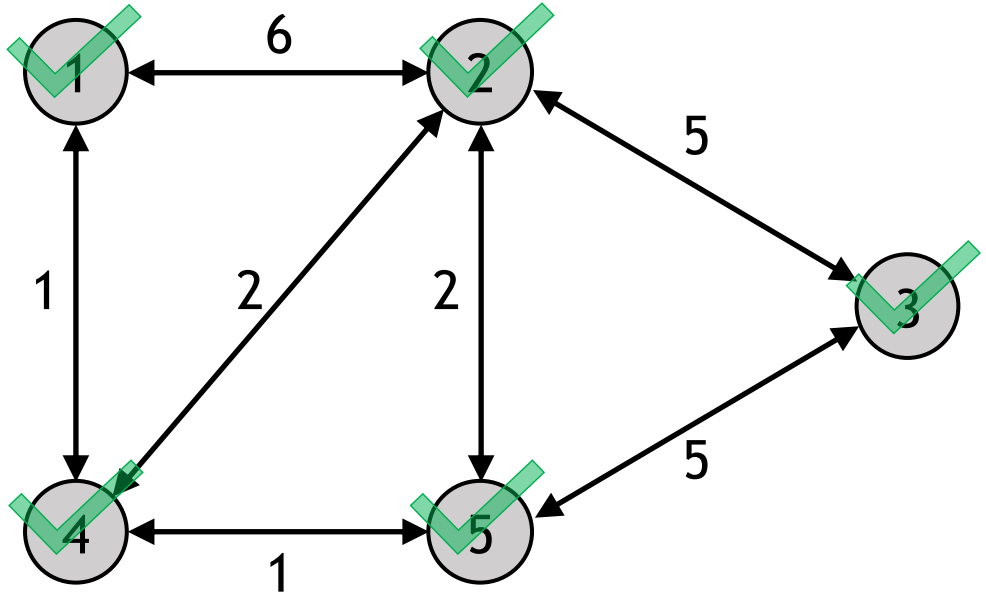
Because there are no 'unvisited' downstream nodes of i, no label update can be made.

Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

$Q = \{\}$

Dijkstra's Shortest Path Algorithm – Step 4 (5)

- If the unvisited node set, Q, is empty (all nodes are finalised), stop. Return $\mathbf{L}^r$ and $\mathbf{q}^r$.
- Otherwise, return to Step 2.

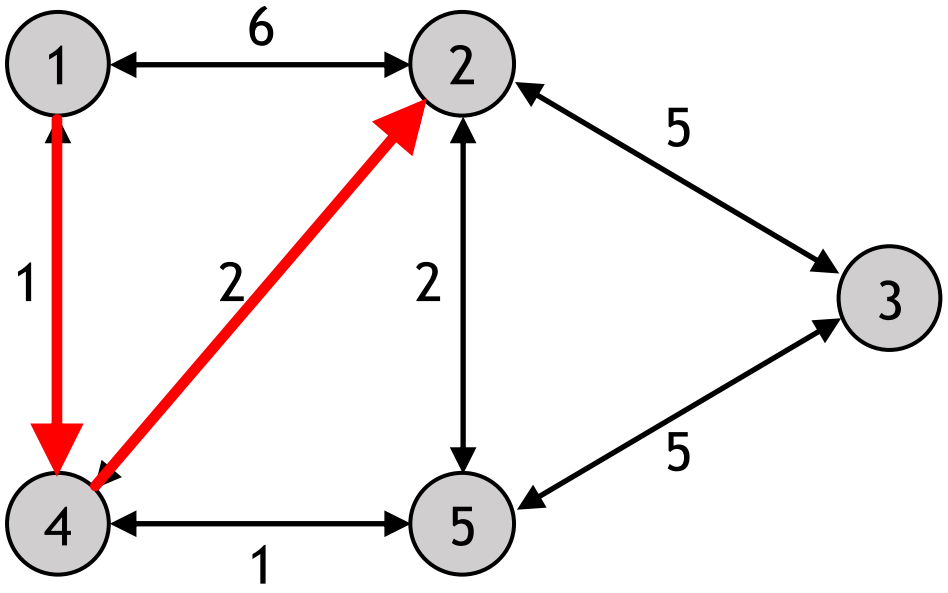


Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

$Q = \{\ \} = \emptyset$
Stop and return $\mathbf{L}^1$ and $\mathbf{q}^1$.

Dijkstra's Shortest Path Algorithm – Output

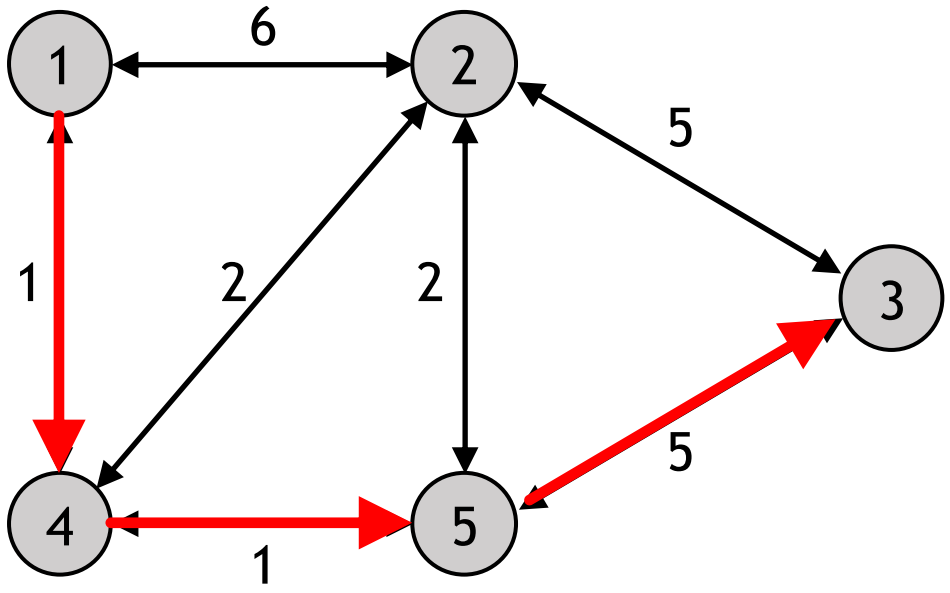
- The shortest path from 1 to 2: $q_2^1 = 4 \rightarrow q_4^1 = 1 \rightarrow q_1^1 = -1$, i.e., Path [1,4,2] with the least travel time of 3



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

Dijkstra's Shortest Path Algorithm – Output

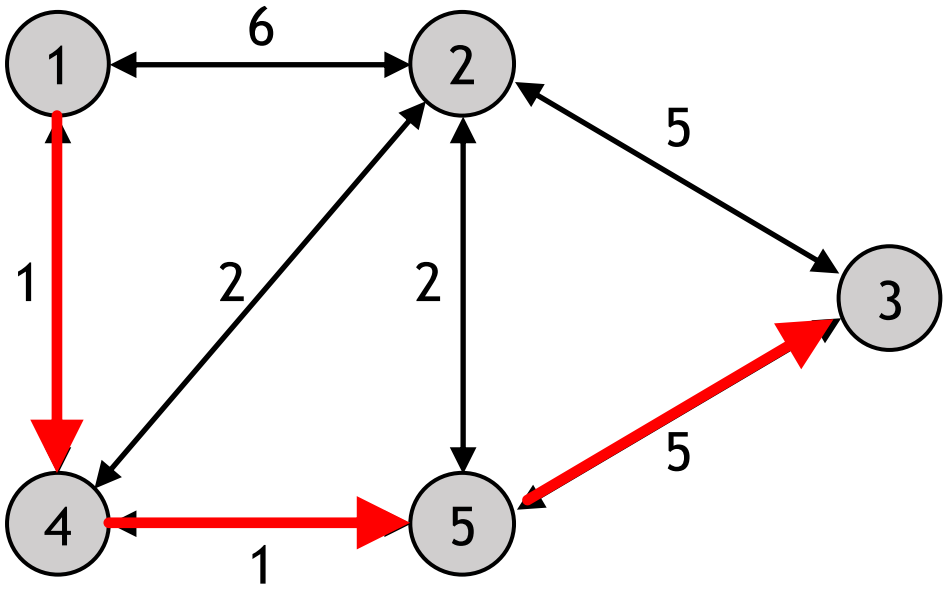
- The shortest path from 1 to 3: $q_3^1 = 5 \rightarrow q_5^1 = 4 \rightarrow q_4^1 = 1 \rightarrow q_1^1 = -1$, i.e., Path [1,4,5,3] with the least travel time of 7



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

Dijkstra's Shortest Path Algorithm – Output

- The shortest path from 1 to 3: $q_3^1 = 5 \rightarrow q_5^1 = 4 \rightarrow q_4^1 = 1 \rightarrow q_1^1 = -1$, i.e., Path [1,4,5,3] with the least travel time of 7

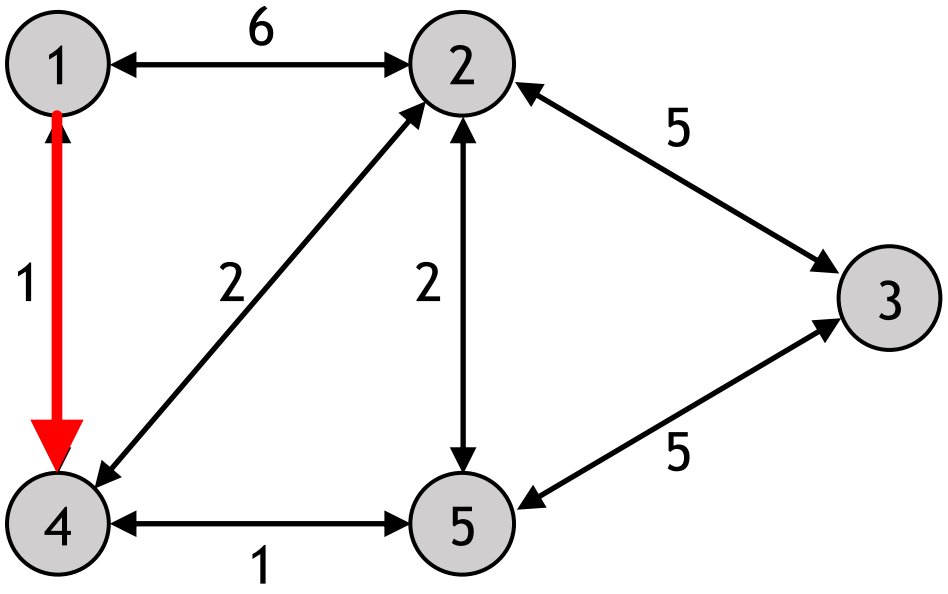


Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

According to the *Optimality Principle*, what would be the shortest paths to 4 and 5 ?

Dijkstra's Shortest Path Algorithm – Output

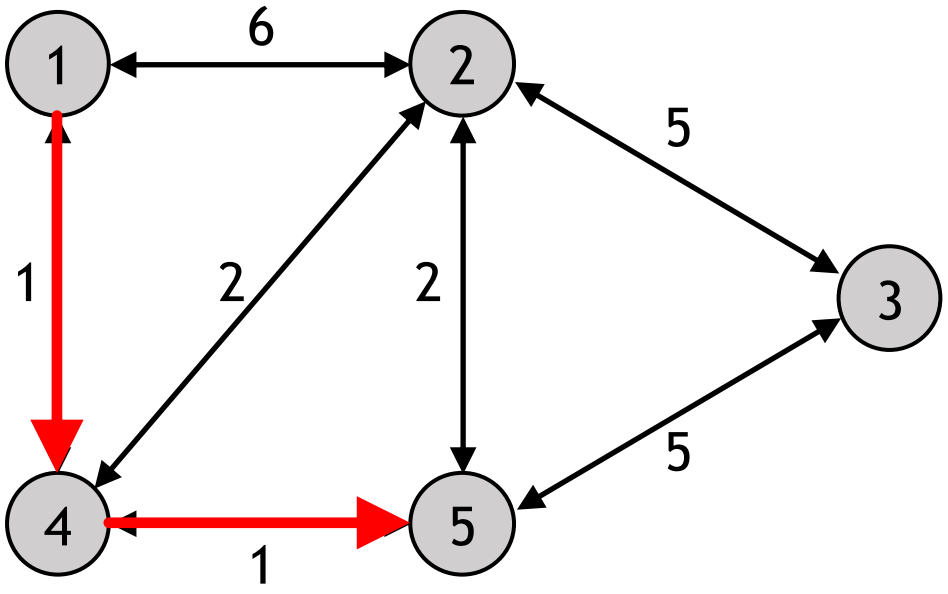
- The shortest path from 1 to 4: $q_4^1 = 1 \rightarrow q_1^1 = -1$, i.e., Path [1,4] with the least travel time of 1



Node i	L_i^1 (current shortest path cost from *r to i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

Dijkstra's Shortest Path Algorithm – Output

- The shortest path from 1 to 5: $q_5^1 = 4 \rightarrow q_4^1 = 1 \rightarrow q_1^1 = -1$, i.e., Path [1,4,5] with the least travel time of 2



Node i	L_i^1 (current shortest path cost from *r* to *i*)	q_i^1 (backnode of *i* in the current shortest path)
1	0	-1
2	3	4
3	7	5
4	1	1
5	2	4

Dijkstra's Shortest Path Algorithm - Summary

1. Initialise every label L_i^r to ∞ , except for the origin. For the origin, set $L_r^r \leftarrow 0$. Initialise the backnode vector $\mathbf{q}^r \leftarrow -1$. Add all nodes to the set of *unvisited* nodes, Q.
2. Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{j \in Q}{\operatorname{argmin}} L_j^r$. Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$.
3. For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows: If $L_i^r + t_{ij} < L_j^r$, $L_j^r \leftarrow L_i^r + t_{ij}$ and $q_j^r \leftarrow i$
4. If the unvisited node set, Q, is empty (all nodes are finalised), stop and return $\mathbf{L}^r$ and $\mathbf{q}^r$. Otherwise, return to Step 2.

Dijkstra's Shortest Path Algorithm - Summary

1. Initialise every label L_i^r to ∞ , except for the origin. For the origin, set $L_r^r \leftarrow 0$. Initialise the backnode vector $\mathbf{q}^r \leftarrow -1$. Add all nodes to the set of *unvisited* nodes, Q.
2. Find an unvisited node $i \in Q$ for which L_i^r is minimal, i.e., $i \leftarrow \underset{i \in Q}{\operatorname{argmin}} L_i^r$. Remove node i from Q, i.e., $Q \leftarrow Q \backslash \{i\}$.
3. For each downstream node j of node i that is still in Q (i.e., $(i,j) \in \Gamma(i)$ and $j \in Q$), update its label and backnode as follows: If $L_i^r + t_{ij} < L_j^r$, $L_j^r \leftarrow L_i^r + t_{ij}$ and $q_j^r \leftarrow i$
4. If the unvisited node set, Q, is empty (all nodes are finalised), stop and return $\mathbf{L}^r$ and $\mathbf{q}^r$. Otherwise, return to Step 2.

Thank you